Kata Tres en Raya: Comparación de Pruebas Unitarias TDD Humano vs. Generadas por IA

Estudiante: Brayan Chambi

Materia: [COMPLETAR MATERIA]

Octubre 2026

Introducción a la Kata y enfoque TDD

- Problema: implementar el juego Tres en Raya (Tic-Tac-Toe) en un tablero de 3x3.
- Reglas: inicia X, los turnos alternan, no se marca una casilla ocupada, coordenadas válidas de 0 a 2.
- Gana quien complete fila, columna o diagonal; si el tablero se llena sin ganador es empate; al terminar no se juega más.
- Ciclo TDD: Rojo (prueba que falla) > Verde (código mínimo) > Refactor.
- Patrón de cada prueba: AAA (Arrange, Act, Assert).

ROJO **VERDE** **REFACTOR**

El prompt utilizado

Actúa como un Ingeniero de Software Senior especialista en Testing y Test-Driven Development (TDD). Genera una suite de pruebas unitarias usando Jest para la Kata del Tres en Raya (Tic-Tac-Toe). Considera las reglas del juego: inicio con turno X, alternancia a O, validación de casillas ocupadas, control de coordenadas fuera de rango (0-2), condiciones de victoria para filas, columnas y diagonales, detección de empate y bloqueo de jugadas cuando la partida terminó. Cada prueba debe estructurarse estrictamente bajo el patrón Arrange-Act-Assert (AAA).

Justificación de los parámetros

- Rol: orienta a la IA a un nivel experto en testing y TDD.
- Framework: Jest, el mismo del proyecto, para que el código sea ejecutable.
- Restricciones de dominio: reglas exactas del juego para evitar casos inventados.
- Patrón AAA: fuerza una estructura uniforme y comparable con las pruebas humanas.

[COMPLETAR: insertar aquí la captura de pantalla de la respuesta de la IA]

Metodología del desarrollo humano

- Una prueba simple a la vez, guiando el diseño de la clase TicTacToe.
- Cada prueba se vio fallar (rojo) antes de escribir el código mínimo para pasarla (verde).
- Orden de prioridad seguido:
  - 1. Estado inicial (turno de X)
  - 2. Alternancia de turno
  - 3. Excepciones: casilla ocupada y fuera de límites
  - 4. Condiciones de victoria: fila, columna y diagonal
  - 5. Empate y fin de partida
- Resultado: 9 pruebas unitarias en Jest, todas en verde, con un commit por cada paso del ciclo.

Metodología del código de la IA

- Generación en lote: la IA entregó todas las pruebas de una sola vez, sin ciclo Rojo-Verde-Refactor.
- Suposición de interfaces: la IA define nombres de métodos y estructuras antes de conocer el código real.
- Resultado observado al ejecutarla contra mi implementación:

[COMPLETAR con tu ejecución real]
- Pruebas generadas: __ - Pasaron sin cambios: __ de __
- Fallos y causa (nombres de métodos, mensajes de error, etc.): _____
- Casos borde extra que propuso la IA: _____

Tabla comparativa de diferencias sustanciales

Aspecto	Pruebas humanas (TDD)	Pruebas generadas por IA
Proceso de desarrollo	Incremental, paso a paso, guiando la arquitectura.	En lote; no aplica el ciclo Rojo-Verde. [Confirmar con tu ejecución]
Aislamiento y granularidad	Cada prueba valida una regla atómica con asserts mínimos.	Tiende a agrupar varios asserts por prueba. [Confirmar]
Manejo de casos borde	Se agregan conforme se identifican durante el refactor.	Anticipa de inmediato entradas nulas, tipos erróneos, etc. [Confirmar]
Mantenibilidad y acoplamiento	Legible y contextualizado al sistema real.	Más genérico; suele requerir ajustar nombres para encajar. [Confirmar]

Las marcas [Confirmar] indican afirmaciones que deben contrastarse con el resultado real de tu prueba con la IA.

Conclusiones y aprendizajes

- La IA es útil como acelerador: propone casos borde y reduce el código repetitivo (boilerplate).
- No sustituye la disciplina del TDD: el ciclo Rojo-Verde-Refactor guía el diseño y da confianza en cada paso.
- Las pruebas de la IA deben revisarse: pueden asumir interfaces que no existen en el código real.
- Lo más efectivo es combinarlos: TDD humano para el diseño, IA para ampliar la cobertura de casos.
- [COMPLETAR: un aprendizaje propio a partir de tu ejecución real]

Enlaces y evidencias

- Repositorio público de GitHub:

https://github.com/[TU_USUARIO]/tictactoe-tdd-kata

- Evidencia de ejecución: captura de la consola con npm test (todas en verde):

[COMPLETAR: pegar aquí la captura de la consola ejecutando npm test]